



AITECHGEN TECHNOLOGIES PVT. LTD.

INTERNSHIP PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

MERN Stack Web Application

Intern: Kruthika S

Internship Domain: MERN Stack / Full-Stack Web Development

Project Duration: 2 Weeks

Internship Period: September 2026

Technology Stack:

MongoDB • Express.js • React.js • Node.js • Axios

TABLE OF CONTENT

Section	Page
1. Project Information	3
2. Internship Context & Project Background	4
3. Abstract	5
4. Introduction	6
5. Problem Statement	7
6. Project Objectives	8
7. Scope of the Project	9
8. Technology Stack	10
9. System Architecture	11
10. Project Structure	12
11. Functional Modules	13
12. Search, Filtering & Pagination	14
13. Database Design	15
14. REST API Design	16–17
15. CRUD Implementation	18–20
16. Validation & Error Handling	21–22
17. User Interface Design	23–24
18. Testing & Results	25–26
19. Development Challenges & Solutions	27
20. Deployment & Live Application	28–29
21. GitHub & Documentation	30–31
22. Development Timeline	32–33
23. Learning Outcomes	34
24. Future Enhancements	35–36
25. Conclusion	37
26. Project Links & References	38

PROJECT INFORMATION

Project Details	Information
Organization	AITechGen Technologies Pvt. Ltd.
Project Title	Student Management System
Internship Domain	MERN Stack/Full-Stack Web Development
Project Type	Internship Mini Project
Project Level	Beginner/ Intermediate
Project Duration	2 Weeks
Internship Period	September 2026
Frontend	React.js with Vite
Backend	Node.js with Express.js
Database	MongoDB Atlas
API Communication	Axios
Version Control	Git & GitHub
Deployment	Render

Project Overview:

The Student Management System is a full-stack web application developed using the MERN stack as part of the internship project at AITechGen Technologies Pvt. Ltd. The application provides a simple interface for managing student records through basic CRUD operations.

The system allows users to add, view, edit, delete, search, and filter student records. React.js is used to build the frontend interface, while Node.js and Express.js handle the backend REST APIs. MongoDB Atlas is used to store student information, and Axios connects the frontend with the backend services.

The project was developed over a two-week mini-project period, covering application setup, frontend development, backend API implementation, database integration, testing, documentation, and deployment.

INTERNSHIP CONTEXT & PROJECT BACKGROUND

1. Internship Context

The Student Management System was developed as a practical mini project during the internship at AITechGen Technologies Pvt. Ltd. The internship provided an opportunity to work with modern web development technologies and apply programming concepts through a complete application development workflow.

The project was carried out in the MERN Stack / Full-Stack Web Development domain, with emphasis on understanding how a frontend application communicates with backend services and how data is stored and managed in a database. The project provided practical exposure to the complete development flow, from setting up the application and implementing features to testing, documentation and deployment.

2. Project Background

As part of the internship, a two-week mini project was assigned to develop a Student Management System using MongoDB, Express.js, React.js, Node.js and Axios. The main purpose of the assignment was to understand CRUD operations and REST API integration using the MERN Stack.

The application was designed to provide a simple and organized way to maintain student records. The required functionality included adding new students, viewing existing records, editing student information, deleting records and searching for specific students.

3. Development Plan

Phase	Activities Completed
Week 1	Project setup, folder structure, React UI, GitHub repository, student form, Axios integration, Express server, MongoDB connection, Add and View functionality
Week 2	Edit, Delete and Search functionality, form validation, application testing, UI improvements, documentation, screenshots and deployment

The assignment specifically divided the development into these two phases, with Week 1 focused on setup and core development and Week 2 on completing CRUD, validation and testing.

4. Project Approach

The application was developed incrementally rather than building all components at once. The frontend interface was first structured using React.js, followed by backend API development with Node.js and Express.js. MongoDB Atlas was then integrated for persistent storage, while Axios was used for communication between the React frontend and REST API.

After implementing the core operations, the application was tested using different student records and user interactions. The final stage focused on improving the interface, preparing documentation and screenshots, and deploying the completed application.

ABSTRACT

The Student Management System is a full-stack web application developed using the MERN Stack as part of an internship mini project at AITechGen Technologies Pvt. Ltd. The project was designed to provide a simple and organized solution for maintaining student records while gaining practical understanding of frontend development, backend services, database operations and REST API integration.

The application allows users to perform the main operations required for student record management, including adding new students, viewing existing records, editing student information, deleting records and searching for students. Course and year-based filtering was also implemented to make the student directory easier to navigate. Basic form validation was included to prevent incomplete records, while a backend uniqueness constraint was used to prevent duplicate Student IDs.

The frontend was developed using React.js with Vite, providing an interactive dashboard and student directory interface. Node.js and Express.js were used to develop the backend REST APIs, while MongoDB Atlas was integrated for persistent storage of student information. Axios was used to establish communication between the frontend and backend.

The completed application was tested using different student records and user interactions to verify the CRUD operations, search, filtering and validation functionality. Git and GitHub were used for version control and project documentation, and the application was deployed using Render.

Through this project, practical experience was gained in developing and connecting the different layers of a full-stack web application, from user interface design and API development to database integration, testing and deployment.

KEYWORDS

MERN Stack • React.js • Node.js • Express.js • MongoDB Atlas • REST API • Axios • CRUD • GitHub • Web Application

INTRODUCTION

1. Introduction to the Project

Student records are an important part of educational administration. Maintaining these records in an organized manner makes it easier to access, update and manage student information whenever required. A web-based Student Management System can simplify these activities by providing a centralized interface for maintaining student details.

The Student Management System developed in this project is a full-stack web application built using the MERN Stack. It provides a structured interface through which student records can be created, viewed, updated, deleted and searched. The application also provides course and year-based filtering to make the student directory easier to navigate.

2. Role of the MERN Stack

The project uses four core technologies of the MERN Stack: MongoDB, Express.js, React.js and Node.js. React.js is responsible for the frontend interface and user interactions. Node.js provides the server-side runtime, while Express.js is used to create the REST API endpoints. MongoDB Atlas provides persistent storage for student records.

Axios is used to send HTTP requests between the React frontend and the Express backend. This separation of frontend, backend and database responsibilities helped provide a clear structure for developing and testing the application.

3. Need for the System

A basic student record management application is useful for demonstrating how information can be maintained through a web interface instead of handling individual records manually. The system provides a single place to perform common operations on student data and reduces the effort involved in locating or modifying records.

The project also addresses an important learning objective of the internship: understanding how different components of a full-stack application work together. From entering information through a React form to storing it in MongoDB and retrieving it through REST APIs, the application demonstrates the complete flow of data across the system.

4. Project Focus

The primary focus of the project was not only to develop the required features but also to understand the development process behind them. The implementation covered frontend design, REST API development, database integration, CRUD operations, search and filtering, validation, testing, version control and deployment.

The project followed a practical progression from initial setup and frontend development to backend integration, complete CRUD functionality, testing, documentation and deployment.

PROBLEM STATEMENT

1. Existing Problem

Managing student information requires maintaining several details such as Student ID, name, email, phone number, course and academic year. When records are handled without a centralized application, adding new information, locating existing records and making updates can become repetitive and difficult to manage efficiently.

A simple record-management application is therefore useful for organizing student information in one place and providing a consistent way to perform common operations on the records.

2. Need for the Proposed System

The project addresses this requirement by providing a web-based Student Management System where student records can be maintained through a single interface. Instead of treating each operation separately, the system connects the user interface, backend services and database so that changes made through the application can be stored and retrieved from a central data source.

The application provides the core operations required for student management: adding new records, viewing existing records, editing information, deleting records and searching for specific students. Course and year filters further help users narrow down the displayed records.

3. Technical Problem to be Addressed

From a development perspective, the project also addresses the challenge of integrating multiple layers of a web application. The frontend must collect and display information, the backend must process requests through REST APIs, and the database must store the records reliably.

The project therefore provides practical implementation of frontend-backend communication, CRUD operations, database integration, validation, error handling and deployment within a single application.

4. Problem Statement

To design and develop a full-stack student management system using the mern stack that provides a simple, centralized and user-friendly interface for creating, viewing, updating, deleting, searching and filtering student records, while demonstrating effective integration between the react frontend, rest api backend and mongodb database.

PROJECT OBJECTIVES

1. Primary Objective

The primary objective of this project was to design and develop a functional Student Management System using the MERN Stack. The application was intended to provide a simple interface for managing student records while demonstrating practical understanding of frontend development, backend API development, database integration and CRUD operations.

2. Specific Objectives

1. To develop a responsive and user-friendly interface using React.js for managing student information.
2. To implement RESTful APIs using Node.js and Express.js for handling student-related operations.
3. To integrate MongoDB Atlas for storing and retrieving student records persistently.
4. To implement CRUD operations for adding, viewing, updating and deleting student records.
5. To provide search functionality based on Student ID, name, email and course.
6. To implement course and academic-year filters for easier navigation of student records.
7. To implement basic validation to prevent incomplete records and handle duplicate Student IDs.
8. To test the application using different records and user interactions to verify the implemented functionality.
9. To use Git and GitHub for version control, project organization and documentation.
10. To deploy the completed application and understand the basic deployment workflow for a full-stack web application.

3. Expected Outcome

The expected outcome was a working full-stack Student Management System in which student information could be created, viewed, updated, deleted and searched through a web interface. The completed system should also demonstrate successful communication between the React frontend, Express/Node.js backend and MongoDB database.

SCOPE OF THE PROJECT

1. Functional Scope

The system focuses on the basic management of student records through a centralized web interface. It provides the required functionality for creating, viewing, updating, deleting and searching student information.

The system includes the following functional areas:

- **Student Record Management:** Add and maintain student details such as Student ID, name, email, phone number, course and academic year.
- **Student Directory:** Display existing student records in a structured tabular format.
- **Search:** Search student records using Student ID, name, email or course.
- **Filtering:** Filter records based on course and academic year.
- **Editing and Deletion:** Update existing student information and remove records when required.
- **Validation:** Prevent incomplete submissions and handle duplicate Student IDs through frontend and backend validation.

2. Technical Scope

The project covers the development of a complete MERN Stack application. React.js is used for the client-side interface, Node.js and Express.js handle the server-side REST APIs, and MongoDB Atlas is used for persistent data storage. Axios connects the frontend with the backend services.

The technical implementation includes:

- **Frontend:** React.js with Vite
- **Backend:** Node.js and Express.js
- **Database:** MongoDB Atlas with Mongoose
- **API Communication:** Axios
- **Version Control:** Git and GitHub
- **Deployment:** Render

3. Development Scope

The project covers the complete development flow from initial setup to deployment. This includes project structure creation, user interface development, REST API implementation, database integration, CRUD functionality, search and filtering, validation, testing, documentation and deployment.

4. Limitations

The current version is intentionally focused on the core requirements of the internship mini project. Features such as user authentication, role-based access, attendance management, marks or grade management, advanced analytics and notification systems are outside the current scope and can be considered for future enhancement.

TECHNOLOGY STACK

1. Frontend Technologies

React.js - React.js was used to build the client-side interface of the Student Management System. It handles the dashboard, student directory, forms and user interactions through reusable components.

Vite - Vite was used as the frontend development and build tool for setting up and running the React application efficiently.

2. Backend Technologies

Node.js - Node.js provides the server-side runtime environment for executing the backend application and handling requests from the frontend.

Express.js - Express.js was used to create the backend REST API and define routes for student-related operations such as retrieving, adding, updating and deleting records.

3. Database

MongoDB Atlas - MongoDB Atlas was used as the cloud database for storing student records. The database provides persistent storage so that records remain available after refreshing or restarting the application.

Mongoose - Mongoose was used as the MongoDB object modeling library to define the Student schema and apply validation rules such as required fields, allowed course and year values, and unique Student IDs.

4. Communication & Development Tools

Axios - Axios was used to send HTTP requests from the React frontend to the Express backend API.

Git & GitHub - Git was used for version control, while GitHub was used to store and manage the project source code and documentation.

Render - Render was used to deploy the frontend and backend so that the completed application could be accessed online.

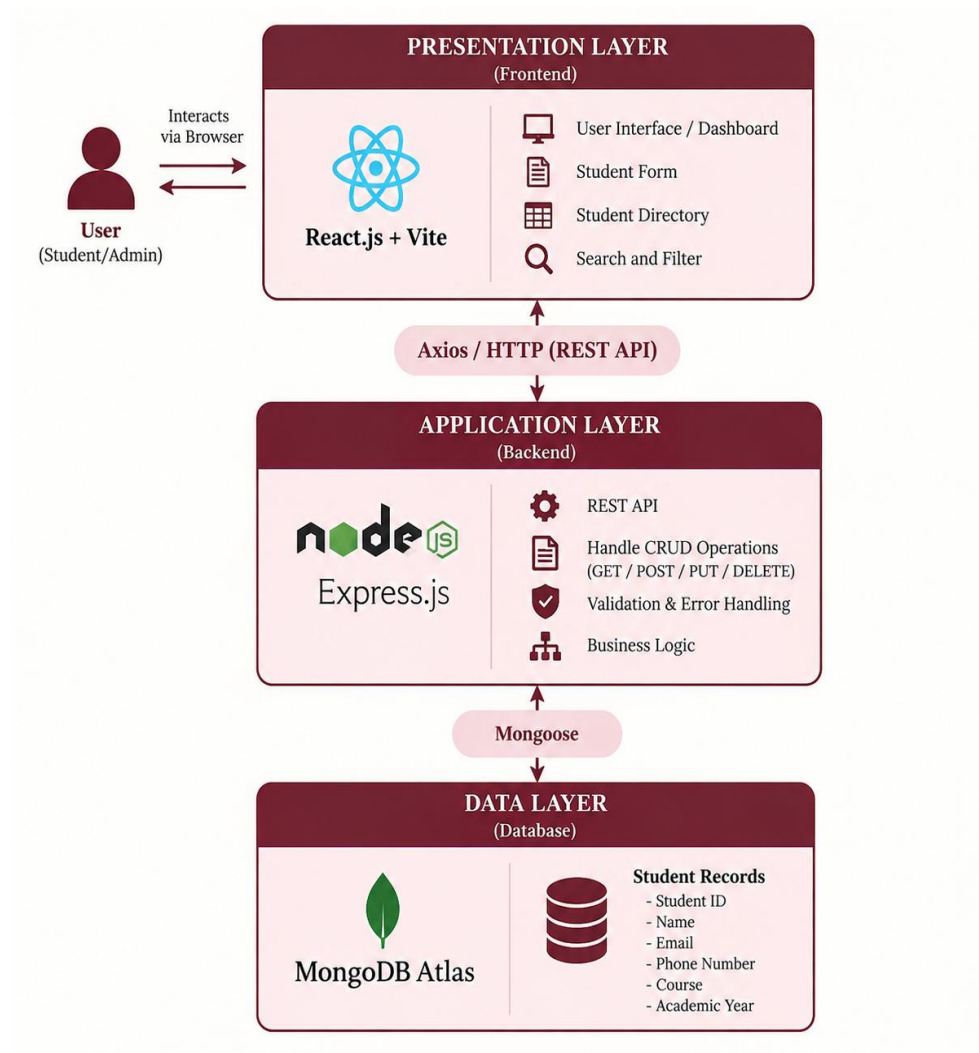
5. Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js + Vite	User interface and interactions
Backend	Node.js + Express.js	REST API and server-side logic
Database	MongoDB Atlas	Persistent student data storage
Data Modeling	Mongoose	Schema and database validation
API Communication	Axios	Frontend-backend communication
Version Control	Git + GitHub	Source code management
Deployment	Render	Hosting the application

SYSTEM ARCHITECTURE

The Student Management System follows a three-layer architecture consisting of the presentation layer, application layer and data layer. The React.js frontend provides the user interface, while the Node.js and Express.js backend processes requests through REST APIs. MongoDB Atlas stores the student records, and Axios is used to communicate between the frontend and backend.

Architecture diagram



Data Flow

1. The user interacts with the React.js frontend through the dashboard, forms and student directory.
2. Axios sends HTTP requests from the frontend to the Express.js REST API.
3. The Node.js and Express.js backend processes the request and communicates with MongoDB through Mongoose.
4. MongoDB Atlas stores or retrieves the required student information.
5. The response is returned through the backend to the React.js interface and displayed to the user.

PROJECT STRUCTURE

The project was organized into separate frontend and backend directories to maintain a clear separation between the user interface, server-side logic and database-related components. This structure makes the application easier to develop, test and maintain.

1. Frontend Structure

```
client/
├── public/
├── src/
│   ├── components/
│   │   ├── Sidebar.jsx
│   │   ├── StatCard.jsx
│   │   ├── StudentDirectory.jsx
│   │   └── StudentForm.jsx
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

The components directory contains the reusable React components used to build the dashboard interface. App.jsx manages the main application state and coordinates communication with the backend API, while index.css contains the styling for the application interface.

2. Backend Structure

```
server/
├── models/
│   └── Student.js
├── routes/
│   └── studentRoutes.js
├── .env
├── package.json
├── package-lock.json
└── server.js
```

The backend is organized into models, routes and the main server file. Student.js defines the MongoDB schema for student records, while studentRoutes.js contains the REST API routes for student operations. server.js configures Express, middleware, MongoDB connectivity and API routing.

3. Root Project Structure

```
student-management-system/
├── client/
├── server/
├── screenshots/
├── .gitignore
├── README.md
├── package.json
└── package-lock.json
```

The root directory contains the frontend, backend and project documentation. The screenshots directory stores application screenshots used for project documentation, while the README provides setup instructions, features, project structure and other repository information.

FUNCTIONAL MODULES

Introduction

The Student Management System is divided into functional modules, with each module responsible for a specific part of the application. These modules work together to provide a complete workflow for managing student records.

1. Dashboard Module

The dashboard provides an overview of the student records available in the system. It displays summary information such as total students, active students, number of courses and recently added records. The dashboard also provides quick access to the student management functions.

2. Student Registration Module

The Add Student module provides a form for entering student information. The form collects Student ID, name, email, phone number, course and academic year. Required-field validation is applied before the information is submitted to the backend.

3. Student Directory Module

The Student Directory displays stored student records in a structured table. Each record includes the Student ID, name, email, course, year and phone number. The module also provides actions for editing and deleting individual records.

4. Search and Filtering Module

The search functionality allows users to find records using Student ID, name, email or course. Course and academic-year filters can also be applied to narrow down the displayed records.

5. Edit Student Module

The Edit Student function allows existing student information to be modified through the same form used for adding records. The updated information is sent to the backend using a PUT request and stored in MongoDB.

6. Delete Student Module

The Delete Student function removes a selected student record from the database. A confirmation prompt is displayed before the deletion request is sent to the backend.

7. Validation and Error Handling Module

The application includes basic validation for required fields and backend validation for the Student schema. Duplicate Student IDs are prevented using a unique database constraint, while API errors are handled and displayed to the user when required.

SEARCH, FILTERING & PAGINATION

1. Search Functionality

The Student Management System provides a search feature to help users quickly locate specific student records. The search can be performed using the Student ID, student name, email address or course.

The search is handled on the frontend using the student records retrieved from the backend. When a search term is entered, the application compares it with the supported student fields and displays only the matching records.

2. Course and Year Filtering

In addition to search, the Student Directory provides filters for course and academic year. Users can select a specific course such as CSE, ECE, EEE or DS and combine it with an academic year from Year 1 to Year 4.

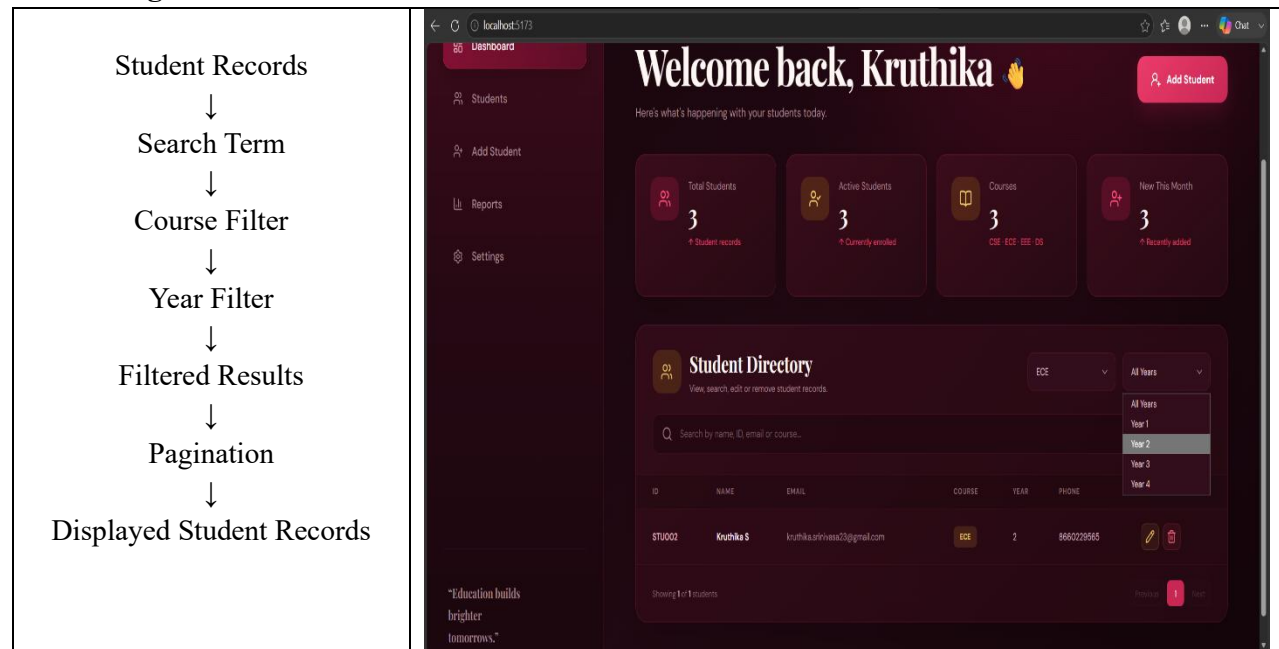
The filters can also be used together with the search functionality, allowing users to narrow down the displayed records based on multiple conditions.

3. Pagination

Pagination was implemented in the Student Directory to prevent too many records from being displayed at once. The application displays a maximum of five student records per page.

Navigation controls allow the user to move between available pages using the Previous and Next buttons or by selecting a specific page number.

4. Filtering Workflow



5. User Benefits

- Faster identification of student records
- Easier navigation through larger datasets
- Combination of search and filters
- Controlled number of records displayed per page
- Improved usability of the Student Directory

DATABASE DESIGN

1. Database Overview

MongoDB Atlas was used as the database for storing student records. The application uses MongoDB's document-based structure, with each student represented as a document in the Student collection. Mongoose is used in the backend to define the schema and manage communication with MongoDB.

2. Student Data Model

Field	Data Type	Required	Validation / Purpose
Student ID	String	Yes	Unique identifier for each student
Name	String	Yes	Student's full name
Email	String	Yes	Stored in lowercase
Phone	String	Yes	Student contact number
Course	String	Yes	CSE, ECE, EEE or DS
Year	String	Yes	Year 1, 2, 3 or 4
Created At	Date	Automatic	Record creation timestamp
Updated At	Date	Automatic	Last modification timestamp

3. Schema Validation

The Student schema uses Mongoose validation rules to maintain consistency in the stored records. Student ID, name, email, phone, course and year are required fields. The Student ID is configured as unique to prevent duplicate records. Course and year values are restricted to the options supported by the application.

4. Database Operations

The backend performs the following operations on the Student collection:

Operation	MongoDB/Mongoose Method	Purpose
Create	Student.save()	Add a new student
Read	Student.find()	Retrieve student records
Read One	Student.findById()	Retrieve a specific student
Update	Student.findByIdAndUpdate()	Modify a student record
Delete	Student.findByIdAndDelete()	Remove a student record

5. Data Flow

When a user submits a student form, the React frontend sends the data to the Express API through Axios. The backend creates or updates the corresponding MongoDB document using Mongoose. When records are requested, the backend retrieves the stored documents and returns them to the frontend for display.

REST API DESIGN

1. API Overview

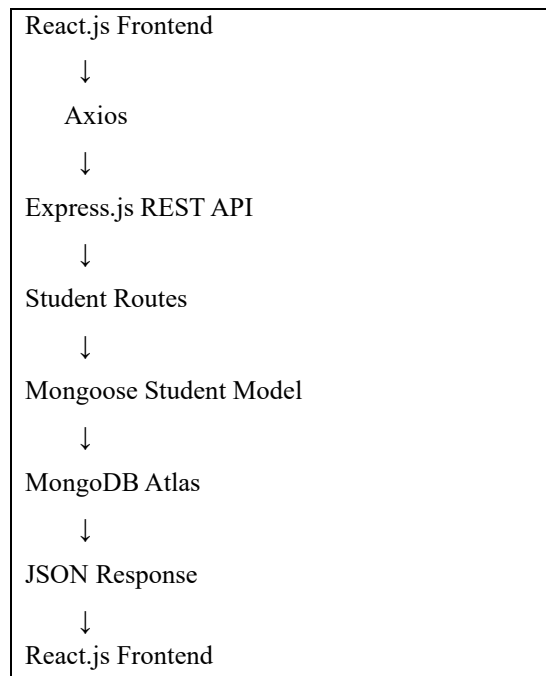
The backend of the Student Management System provides a RESTful API for managing student records. The API is implemented using Node.js and Express.js and follows standard HTTP methods for performing CRUD operations. Axios is used by the React frontend to communicate with these endpoints.

2. API Endpoints

Method	Endpoint	Purpose
GET	/api/students	Retrieve all student records
GET	/api/students/:id	Retrieve a specific student
POST	/api/students	Add a new student
PUT	/api/students/:id	Update an existing student
DELETE	/api/students/:id	Delete a student record

3. API Request Flow

When an operation is performed from the React interface, Axios sends an HTTP request to the corresponding Express route. The route processes the request and communicates with MongoDB through the Mongoose Student model. The server then returns the result as a JSON response to the frontend.



4. HTTP Methods Used

GET

Used to retrieve student records from the database.

POST

Used to create and store a new student record.

PUT

Used to update an existing student record.

DELETE

Used to permanently remove a selected student record.

5. API Response and Error Handling

The API returns student data in JSON format when requests are successful. Appropriate HTTP status codes are used for common outcomes such as successful creation, missing records and invalid requests. Backend errors are caught and returned with a descriptive message so that the frontend can display appropriate feedback.

6. Example Endpoint

```
POST /api/students
```

```
Request:
```

```
{
  "studentId": "STU005",
  "name": "Example Student",
  "email":
"student@example.com",
  "phone": "9876543210",
  "course": "CSE",
  "year": "3"
}
```

CRUD IMPLEMENTATION

1. CRUD Overview

CRUD stands for Create, Read, Update and Delete. These four operations form the core functionality of the Student Management System. Each operation is implemented through the React frontend, Express.js REST API and MongoDB database.

2. Create Operation

The Create operation is performed when a user submits the Add Student form. The React frontend collects the student details and sends them to the backend using an HTTP POST request. The Express route creates a new Student document and saves it to MongoDB using Mongoose.

Flow:

Add Student Form → Axios → POST /api/students → Mongoose → MongoDB

3. Read Operation

The Read operation retrieves student records from MongoDB. When the application loads, the frontend sends a GET request to the backend. The server retrieves the records using the Student model and returns them as JSON data for display in the Student Directory.

Flow:

Student Directory → Axios → GET /api/students → MongoDB → JSON Response

4. Update Operation

The Update operation allows an existing student record to be edited. When the user selects the edit action, the student's existing information is loaded into the form. After the changes are submitted, a PUT request is sent to the backend and the corresponding MongoDB document is updated.

Flow:

Edit Form → Axios → PUT /api/students/:id → MongoDB → Updated Record

5. Delete Operation

The Delete operation removes an existing student record. A confirmation prompt is displayed before the request is submitted. The frontend sends a DELETE request to the corresponding backend route, which removes the selected document from MongoDB.

Flow:

Delete Action → Confirmation → DELETE /api/students/:id → MongoDB

6. CRUD Summary

Operation	HTTP Method	Endpoint	Result
Create	POST	/api/students	New student added
Read	GET	/api/students	Student records retrieved
Update	PUT	/api/students/:id	Student record modified
Delete	DELETE	/api/students/:id	Student record removed

Figure 15.1. Add Student form for creating a new student record

Figure 15.2. Student Directory displaying stored student records

The screenshot shows a web browser window with the address bar displaying 'localhost:5173'. The main content area features a dark-themed modal window titled 'UPDATE RECORD' and 'Edit Student'. Below the title is the instruction 'Update the student's information.' The form contains several input fields: 'Student ID' with the value 'STU003', 'Full Name' with 'ANAYA', 'Email' with 'anayagani@dsu.edu.in', and 'Phone' with '9876543210'. There are also dropdown menus for 'Course' (set to 'CSE') and 'Year' (set to 'Year 1'). At the bottom right of the form are two buttons: a grey 'Cancel' button and a red 'Update Student' button with a person icon.

Figure 15.3. Edit Student form for updating an existing student record

The screenshot shows a web browser window with the address bar displaying 'localhost:5173'. The main content area features a dark-themed modal window titled 'NEW RECORD' and 'Add New Student'. Below the title is the instruction 'Enter the student's details below.' A red error message 'Please fill in all fields.' is displayed above the input fields. The form contains several input fields: 'Student ID' with a placeholder 'e.g. STU006', 'Full Name' with a placeholder 'Enter full name', 'Email' with a placeholder 'student@example.com', and 'Phone' with a placeholder 'Enter phone number'. There are also dropdown menus for 'Course' (set to 'Select course') and 'Year' (set to 'Select year'). At the bottom right of the form are two buttons: a grey 'Cancel' button and a red 'Add Student' button with a person icon.

Figure 15.4. Form validation and duplicate Student ID handling

VALIDATION & ERROR HANDLING

1. Validation Overview

Validation was implemented to ensure that student records contain the required information before they are stored in the database. Validation is handled at both the frontend and backend levels, providing an additional layer of data consistency.

2. Frontend Validation

The React student form checks whether all required fields have been completed before submitting the request. The required fields are Student ID, name, email, phone number, course and academic year. If any field is empty, the form displays an error message and prevents submission.

3. Backend Validation

The Student schema defined using Mongoose contains validation rules for the required fields. Course and academic year values are restricted to the options supported by the application. Email values are converted to lowercase before being stored.

4. Duplicate Student ID Handling

Student IDs are configured as unique in the Mongoose schema. If a user attempts to create another record with an existing Student ID, the backend identifies the duplicate-key error and returns a descriptive message indicating that the Student ID already exists.

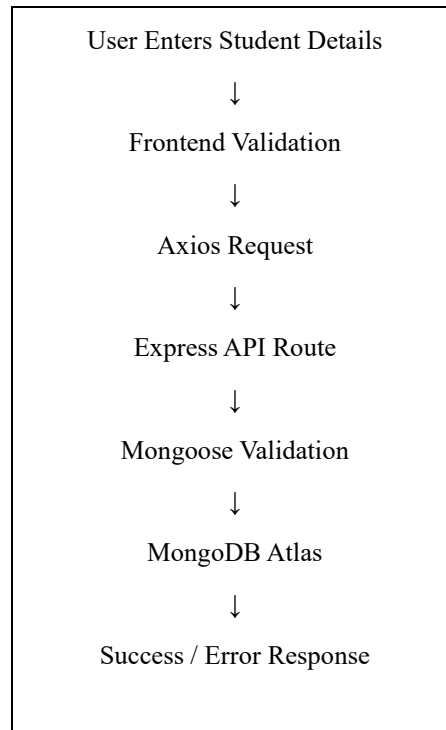
5. API Error Handling

The Express routes use try...catch blocks to handle errors during database operations. Depending on the operation, the API returns an appropriate HTTP status code along with a descriptive error message. These responses are handled by the frontend and displayed when an operation cannot be completed.

6. Delete Confirmation

Before deleting a student record, the application displays a confirmation prompt. This provides an additional check before a record is permanently removed from the database.

7. Validation Flow



8. Validation Evidence

The screenshot shows a web browser window with the address bar displaying 'localhost:5173'. The main content is a dark-themed modal form titled 'Add New Student' with a subtitle 'Enter the student's details below.' and a close button (X) in the top right corner. The form includes a message 'Please fill in all fields.' and several input fields:

- Student ID:** A text input field with the placeholder 'e.g. STU006'.
- Full Name:** A text input field with the placeholder 'Enter full name'.
- Email:** A text input field with the placeholder 'student@example.com'.
- Phone:** A text input field with the placeholder 'Enter phone number'.
- Course:** A dropdown menu with the placeholder 'Select course'.
- Year:** A dropdown menu with the placeholder 'Select year'.

At the bottom right of the form, there are two buttons: a 'Cancel' button and an 'Add Student' button with a plus icon.

Figure 16.1. Student form validation and duplicate Student ID handling

USER INTERFACE DESIGN

1. Design Approach

The user interface was designed with a focus on simplicity, readability and ease of navigation. A dashboard-based layout was used to provide quick access to student records and management functions from a single interface.

The application follows a Burgundy Luxe visual theme with burgundy backgrounds, light typography, subtle gold accents and rounded cards. The design was kept consistent across the dashboard, student directory and student forms.

2. Dashboard

The dashboard acts as the main landing page of the application. It provides an overview of student records through summary cards and gives direct access to the Add Student and Student Directory features.

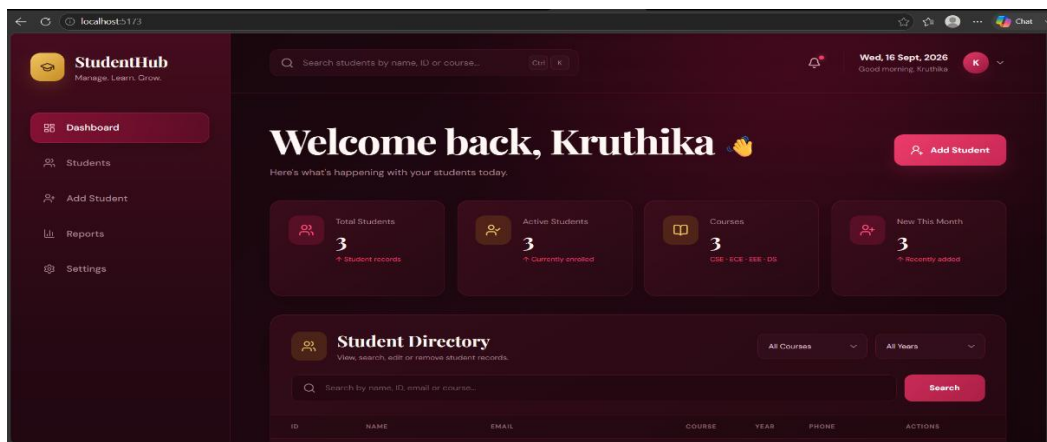


Figure 17.1. Student Management System dashboard

3. Add Student Interface

The Add Student form provides a structured interface for entering student information. Input fields are provided for Student ID, name, email, phone, course and academic year. The form also provides clear actions for saving or cancelling the operation.

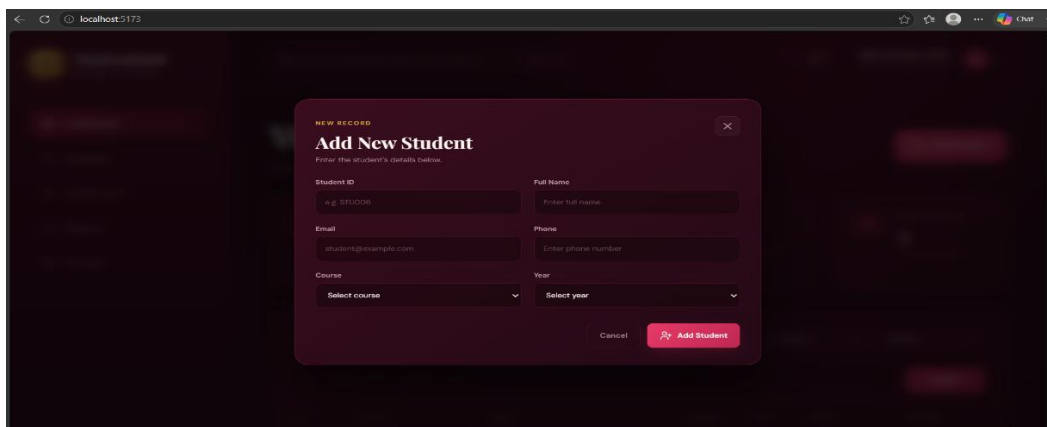


Figure 17.2. Add Student form interface

4. Student Directory

The Student Directory presents student records in a tabular format. Search, course and year filters are provided to make records easier to locate. Edit and delete actions are available for individual records.

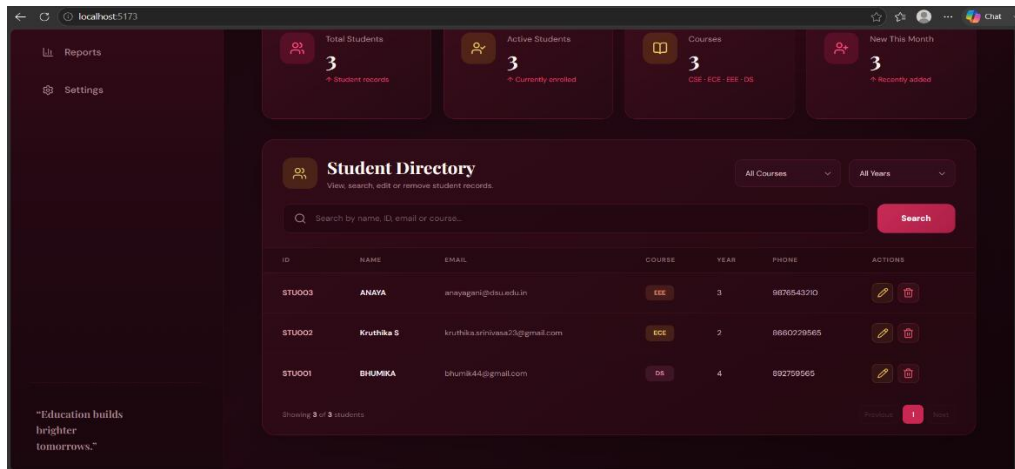


Figure 17.3. Student Directory interface

5. UI Design Principles

The interface follows these principles:

- **Consistency:** Common colors, typography and spacing are maintained throughout the application.
- **Clarity:** Important information and actions are visually separated.
- **Navigation:** The sidebar provides access to the main application sections.
- **Feedback:** Validation messages and confirmation prompts inform users about important actions.
- **Responsiveness:** The layout is structured to adapt to different screen sizes.

TESTING & RESULTS

1. Testing Approach

Testing was performed after implementing the major application features to verify that the frontend, backend API and database operations worked together correctly. Different student records and user interactions were used to check the core functionality of the application.

2. Functional Testing

Test Case	Action Performed	Expected Result	Status
Add Student	Enter valid student details and submit	New record is added and displayed	Passed
View Students	Open Student Directory	Stored records are displayed	Passed
Edit Student	Modify an existing student record	Updated information is displayed	Passed
Delete Student	Confirm deletion of a record	Selected record is removed	Passed
Search by Name	Search using student name	Matching records are displayed	Passed
Search by ID	Search using Student ID	Matching record is displayed	Passed
Search by Course	Search using course name	Matching records are displayed	Passed
Course Filter	Select a course	Records are filtered by course	Passed
Year Filter	Select academic year	Records are filtered by year	Passed
Combined Filter	Apply course and year filters	Matching records are displayed	Passed
Empty Field Validation	Submit incomplete form	Submission is prevented with an error	Passed
Duplicate Student ID	Enter an existing Student ID	Duplicate ID error is displayed	Passed
Delete Confirmation	Select delete action	Confirmation prompt appears	Passed
Data Persistence	Refresh application after adding a record	Record remains available	Passed

3. Integration Testing

Integration testing was used to verify communication between the React frontend, Axios requests, Express.js API routes and MongoDB Atlas. CRUD operations were tested through the actual application interface to confirm that changes made by the user were reflected in the database and returned correctly to the frontend.

4. Deployment Testing

After deployment, the frontend and backend were tested through their deployed environments to verify that the application could load correctly and communicate with the backend API. MongoDB Atlas connectivity was also verified as part of the deployment process.

5. Test Result

The implemented test cases produced the expected results. The core requirements of the Student Management System—including CRUD operations, search, filtering, validation and data persistence—were successfully verified.

DEVELOPMENT CHALLENGES & SOLUTIONS

1. Frontend–Backend Integration

One of the main challenges was connecting the React frontend with the Express.js backend and ensuring that requests were sent to the correct API endpoints. Axios was used to establish communication between the two layers, and each CRUD operation was tested individually to identify request or response issues.

2. MongoDB Atlas Connectivity

Connecting the backend to MongoDB Atlas required configuring the database connection correctly and ensuring that the application could access the database. The MongoDB connection string was stored in the local .env file rather than being included directly in the source code.

3. CRUD Synchronization

Another challenge was keeping the frontend student list synchronized with changes made to the database. After adding, editing or deleting a record, the React state was updated so that the displayed records reflected the latest data.

4. Search and Filtering

Implementing search together with course and academic-year filters required the different conditions to work together correctly. The filtering logic was designed to check the search term and selected filters before determining which records should be displayed.

5. Form Validation

The application needed to prevent incomplete student records and duplicate Student IDs. Required-field validation was implemented on the frontend, while Mongoose validation and the unique Student ID constraint provided backend-level protection.

6. Deployment Configuration

During deployment, the backend initially faced a database connectivity issue because the deployed service could not access MongoDB Atlas. The database network access configuration was updated to allow the deployed backend to establish the required connection.

7. UI and Layout Refinement

Considerable attention was given to improving the dashboard layout, spacing, forms, filters and student table. The interface was refined to maintain a consistent Burgundy Luxe theme while keeping the main student management functions easy to access.

8. Key Learning from Challenges

These challenges provided practical experience in debugging a full-stack application rather than working with each technology separately. Resolving them helped in understanding API communication, database connectivity, frontend state management, validation and deployment configuration.

DEPLOYMENT & LIVE APPLICATION

1. Deployment Overview

After completing development and testing, the Student Management System was deployed so that the application could be accessed outside the local development environment. Render was used to host both the frontend and backend, while MongoDB Atlas continued to provide cloud-based database storage.

2. Backend Deployment

The Node.js and Express.js backend was deployed as a web service on Render. The deployed backend provides the REST API required by the React frontend for student record operations.

Live Backend API: [Student Management System – REST API](#)

3. Frontend Deployment

The React.js frontend was deployed as a static site on Render. The frontend was configured to communicate with the deployed backend API instead of the local development server.

Live Application: [Student Management System – Live Demo](#)

4. Deployment Architecture

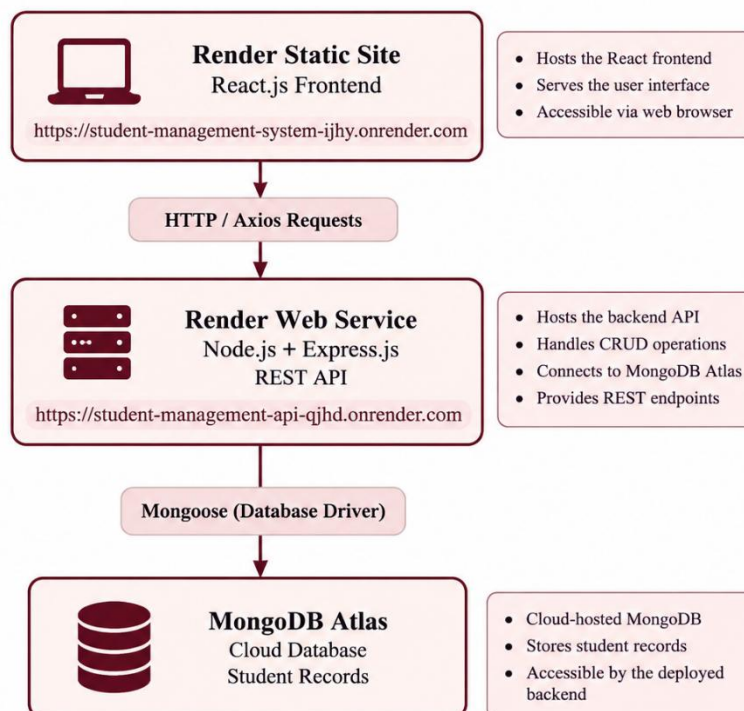


Figure 20.1. Deployment Architecture of the Student Management System

5. Deployment Configuration

Environment-specific configuration such as the MongoDB connection string was kept outside the source code using environment variables. The .env file was excluded from GitHub through .gitignore, while the required environment variable was configured separately for the deployed backend.

6. Deployment Verification

After deployment, the live frontend was opened and tested to verify that the dashboard loaded correctly and that student records could be retrieved through the deployed backend. CRUD operations, search, filtering and validation were also verified after connecting the application to the deployed services.

7. Deployment Challenge

During the initial backend deployment, the deployed service could not connect to MongoDB Atlas because database network access had not yet been configured for the deployment environment. The MongoDB Atlas network access settings were updated, after which the backend successfully connected to the database and the deployment became operational.

GITHUB & DOCUMENTATION

1. Version Control

Git was used throughout the development process to track changes to the project source code. The project was maintained using a Git repository with the main branch used for the completed application.

2. GitHub Repository

GitHub was used to host the source code and provide a central location for the project files, documentation and screenshots. The repository contains the frontend, backend, project configuration files, README documentation and selected project screenshots.

GitHub Repository: [Student Management System – GitHub Repository](#)

3. Repository Structure

The final repository is organized as follows:

```
student-management-system/  
|  
├─ client/  
|   └─ React frontend  
|  
├─ server/  
|   ├─ models/  
|   ├─ routes/  
|   └─ server.js  
|  
├─ screenshots/  
|   ├─ 01-dashboard.png  
|   ├─ 02-add-student.png  
|   ├─ 03-student-directory.png  
|   ├─ 04-search-filter.png  
|   ├─ 05-edit-student.png  
|   └─ 06-validation.png  
|  
├─ .gitignore  
├─ README.md  
├─ package.json  
└─ package-lock.json
```

4. README Documentation

A README file was prepared to document the project and make the repository easier to understand. It includes the project overview, objectives, features, technology stack, architecture, project structure, CRUD operations, REST API information, database model, validation, testing, setup instructions, screenshots, learning outcomes and future enhancements.

5. Security and Repository Practices

Environment-specific credentials were kept outside the source code. The backend .env file containing the MongoDB connection string was excluded from GitHub using .gitignore. Node modules were also excluded from the repository to avoid unnecessarily storing generated dependency files.

6. Screenshots and Project Evidence

Selected screenshots were added to the repository to document the completed application and provide visual evidence of the implemented functionality. These include the dashboard, student form, directory, search and filtering, editing and validation features.

DEVELOPMENT TIMELINE

1. Project Development Plan

The Student Management System was developed over a two-week mini-project period. Development was divided into stages so that the frontend, backend, database integration and application features could be implemented and tested progressively.

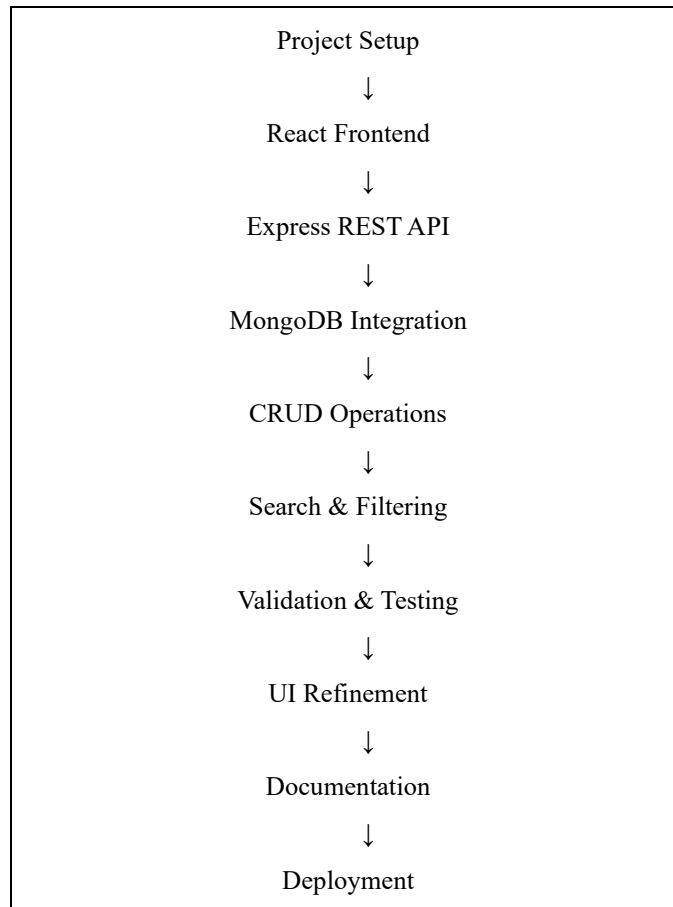
2. Week 1 — Setup & Core Development

Stage	Work Completed
Project Setup	Created the MERN project structure and configured the frontend and backend
Frontend Development	Built the React dashboard, sidebar, student form and student directory
Version Control	Created the GitHub repository and maintained the project using Git
Backend Development	Set up Node.js and Express.js and created the REST API
Database Integration	Connected the backend to MongoDB Atlas using Mongoose
Create & Read	Implemented Add Student and View Students functionality

3. Week 2 — Features, Testing & Deployment

Stage	Work Completed
Update	Implemented editing of existing student records
Delete	Implemented deletion with confirmation
Search	Added search by Student ID, name, email and course
Filtering	Added course and academic-year filters
Validation	Added required-field and duplicate Student ID handling
Testing	Tested CRUD operations, search, filters, validation and persistence
UI Refinement	Improved dashboard layout, styling and overall usability
Documentation	Prepared README, screenshots and internship report content
Deployment	Deployed frontend and backend using Render

4. Development Workflow



5. Project Completion

The development process progressed from basic project setup to a complete deployed application. Each major feature was tested after implementation, allowing issues in frontend interaction, API communication and database operations to be identified and resolved before the final deployment.

LEARNING OUTCOMES

1. Full-Stack Development

The project provided practical experience in developing a complete web application using the MERN Stack. It helped in understanding how the frontend, backend and database layers work together as a single application.

2. React.js Development

Working with React.js provided experience in building reusable components, managing application state and creating interactive interfaces for student record management.

3. REST API Development

The project provided practical understanding of creating REST API endpoints using Node.js and Express.js and connecting frontend actions to backend operations through Axios.

4. Database Management

MongoDB Atlas and Mongoose were used to store and manage student records. The project provided experience with schema design, required fields, unique constraints and database CRUD operations.

5. API and Database Integration

Connecting the React frontend, Express backend and MongoDB database helped develop an understanding of how data moves through different layers of a full-stack application.

6. Validation and Error Handling

Implementing required-field validation, duplicate Student ID handling and API error responses provided practical experience in maintaining data consistency and handling unexpected situations.

7. Git and GitHub

The project strengthened understanding of Git-based version control and GitHub repository management, including organizing source code, maintaining documentation and managing project files.

8. Deployment

Deploying the application using Render provided practical exposure to moving a locally developed application into an online environment and configuring communication between the deployed frontend, backend and database.

9. Problem-Solving Skills

The development process involved identifying and resolving issues related to API communication, database connectivity, frontend state updates, validation and deployment. This helped improve systematic debugging and problem-solving skills.

10. Overall Learning

Overall, the project provided practical experience across the complete application lifecycle, from initial planning and implementation to testing, documentation and deployment. It also helped connect theoretical programming concepts with a working full-stack application.

FUTURE ENHANCEMENTS

1. User Authentication

User authentication can be added to restrict access to the Student Management System and ensure that only authorized users can manage student records.

2. Role-Based Access

Different access levels can be introduced for administrators, faculty members and other users. Each role could be provided with permissions appropriate to its responsibilities.

3. Student Academic Information

The system can be extended to store additional academic information such as marks, grades, attendance and semester-wise performance.

4. Advanced Reporting

Reporting features can be added to generate summaries of student records based on course, academic year and other available attributes.

5. Improved Dashboard Analytics

The dashboard can be enhanced with charts and visual analytics to provide a clearer overview of student data and trends.

6. Advanced Search

The search functionality can be expanded to support more advanced combinations of search criteria and additional student attributes.

7. Notifications

Notification functionality could be introduced for important events such as newly added records, updates or other administrative activities.

8. Automated Testing

Automated unit and integration tests can be added to improve regression testing and make future modifications safer.

9. Cloud and Infrastructure Improvements

The deployed application can be further improved with production-oriented configuration, monitoring and automated deployment workflows.

10. Mobile-Friendly Enhancements

The interface can be further optimized for mobile devices so that student records can be accessed conveniently across different screen sizes.

Current Scope vs Future Scope

Current Implementation	Possible Future Enhancement
Student record CRUD	Authentication and authorization
Course and year filtering	Advanced analytics
Basic validation	Expanded validation and automated testing
Dashboard statistics	Interactive reports and charts
Render deployment	Production monitoring and CI/CD

Closing Statement

These enhancements are outside the scope of the current two-week internship mini project. They represent possible directions for extending the application into a more comprehensive student information management platform.

CONCLUSION

1. Project Summary

The Student Management System was successfully developed as a full-stack internship mini project using the MERN Stack. The application provides a centralized interface for managing student records and implements the core requirements of the assigned project.

2. Functional Completion

The completed system supports the major student management operations, including adding, viewing, editing, deleting and searching student records. Course and academic-year filtering, pagination, form validation and duplicate Student ID handling were also implemented.

3. Technical Implementation

The project demonstrates integration between React.js, Node.js, Express.js and MongoDB Atlas. Axios was used for communication between the frontend and backend, while Mongoose was used for database schema definition and validation. Git and GitHub were used for version control and documentation.

4. Testing and Deployment

The application was tested using different student records and user interactions to verify CRUD operations, search, filtering, validation and data persistence. The completed frontend and backend were deployed using Render, with MongoDB Atlas providing the cloud database.

5. Internship Learning

Developing this project provided practical experience with the complete lifecycle of a web application. The work involved planning, frontend development, backend API implementation, database integration, debugging, testing, documentation and deployment.

6. Final Outcome

The project successfully transformed the requirements of a basic Student Management System into a working web application. Beyond implementing the required features, the development process provided a practical understanding of how different technologies and application layers work together to deliver a functional full-stack solution.

Closing Statement

Overall, the internship project provided a valuable opportunity to apply software development concepts in a practical environment and build a complete application from initial setup through final deployment.

PROJECT LINKS & REFERENCES

1. Project Repository

GitHub Repository: [Student Management System – GitHub Repository](#)

2. Live Application

Live Application: [Student Management System – Live Demo](#)

3. Backend API

Backend API: [Student Management System – REST API](#)

4. Technologies & Documentation

React.js Documentation: <https://react.dev/>

Node.js Documentation: <https://nodejs.org/docs/latest/api/>

Express.js Documentation: <https://expressjs.com/>

MongoDB Documentation: <https://www.mongodb.com/docs/>

Mongoose Documentation: <https://mongoosejs.com/docs/>

Axios Documentation: <https://axios-http.com/docs/intro>

Vite Documentation: <https://vite.dev/guide/>

GitHub Documentation: <https://docs.github.com/>

Render Documentation: <https://render.com/docs>

5. Internship Assignment Reference

AITechGen Technologies Pvt. Ltd. — Internship Mini Project Assignment: MERN Stack Student Management System.

6. Project Documentation

Project README, source code, screenshots and implementation details are maintained in the GitHub repository.